CLOUD COMPUTING LAB

Course Code: CSE12045

EXPERIMENT 5

Implementation of Various Scheduling Mechanisms Using an Open-Source Cloud Simulator (CloudSim Plus)

Name	Sk Naser Hussain
Roll Number	UG/02/BTCSE/2023/151
Section	B
Course	Cloud Computing Lab (CSE12045)
Date of Experiment	29/10/2026

1. Aim

To implement and study the different scheduling mechanisms available in a cloud environment - the VM scheduler (host level) and the cloudlet scheduler (VM level) - using the open-source CloudSim Plus simulator, and to compare the four combinations of space-shared and time-shared policies.

2. Objectives

• Set up a Java 17 and Apache Maven based development environment on Ubuntu.
• Create a Maven project that uses CloudSim Plus 8.5.5 as a dependency.
• Build a simulated datacenter with one host, two VMs and four cloudlets.
• Run four scheduling scenarios (A, B, C, D) and observe the start and finish times of the cloudlets.

3. Theory

CloudSim Plus is an open-source Java framework for modelling and simulating cloud computing infrastructure and services. It lets us model datacenters, hosts, virtual machines (VMs) and cloudlets (tasks) without needing real hardware.

Scheduling happens at two levels:

VM Scheduler (host level) decides how the physical processing elements (PEs / cores) of a host are given to the VMs placed on it.
• *VmSchedulerSpaceShared*: each PE is dedicated to one VM; a VM can start only if free PEs exist.
• *VmSchedulerTimeShared*: the PEs' capacity is shared among VMs over time, so more VMs than physical PEs can be hosted (each getting a fraction of the capacity).

Cloudlet Scheduler (VM level) decides how a VM's PEs are given to the cloudlets running inside it.
• *CloudletSchedulerSpaceShared*: a cloudlet gets its PE(s) exclusively; extra cloudlets wait in a queue until a PE becomes free.
• *CloudletSchedulerTimeShared*: all cloudlets run together and share the VM's capacity in time slices.

Combining the two levels gives four scenarios:

Scenario	VM Scheduler	Cloudlet Scheduler
A	Space-Shared	Space-Shared
B	Space-Shared	Time-Shared
C	Time-Shared	Space-Shared
D	Time-Shared	Time-Shared

4. Simulation Parameters

Parameter	Value
Host	1 host, 4 PEs x 1000 MIPS, 8192 MB RAM, 100000 bandwidth, 1000000 storage
Virtual Machines	2 VMs, each 2 PEs x 1000 MIPS, 2048 MB RAM, 10000 bandwidth, 10000 size
Cloudlets	4 cloudlets, each 20000 MI length, 1 PE, UtilizationModelFull, 300 file size
Broker	DatacenterBrokerSimple
Software	Ubuntu (VirtualBox), OpenJDK 17.0.20.1, Maven 3.8.7, Git 2.43.0, CloudSim Plus 8.5.5

5. Procedure and Implementation

Step 1: Update Ubuntu

The package index is refreshed so that Ubuntu knows the latest available package versions, and then installed software is upgraded. The *-y* flag confirms the operation automatically.

```
sudo apt update
```

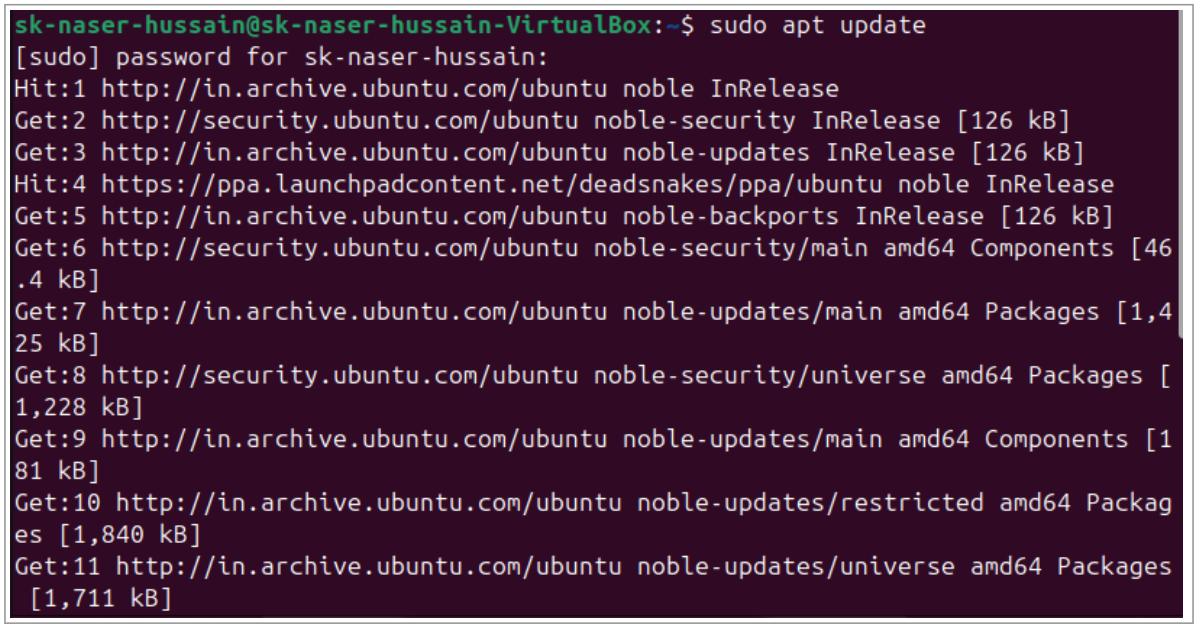


Fig. 1: Output of sudo apt update - package lists fetched from the Ubuntu repositories

```
sudo apt upgrade -y
```

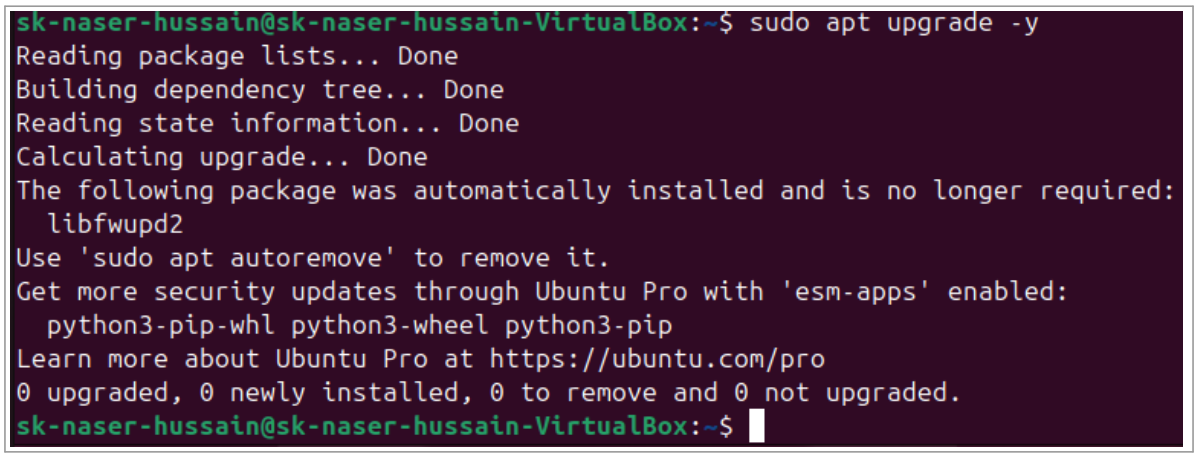


Fig. 2: Output of sudo apt upgrade -y - system already up to date (0 upgraded)

Step 2: Install Java, Maven and Git

OpenJDK 17 (compiler and runtime), the Maven build tool and Git are installed in a single command.

```
sudo apt install openjdk-17-jdk maven git -y
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ sudo apt install openjdk-17-jdk
maven git -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libfwupd2
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  ca-certificates-java fonts-dejavu-extra git-man java-common
  libaopalliance-java libapache-pom-java libatinject-jsr330-api-java
  libatk-wrapper-java libatk-wrapper-java-jni libcdi-api-java
  libcommons-cli-java libcommons-io-java libcommons-lang3-java
  libcommons-parent-java liberror-perl liberror-prone-java
  libgeronimo-annotation-1.3-spec-java libgeronimo-interceptor-3.0-spec-java
  libguava-java libguice-java libice-dev libjansi-java libjsr305-java
  libmaven-parent-java libmaven-resolver-java libmaven-shared-utils-java
  libmaven3-core-java libplexus-cipher-java libplexus-classworlds-java
  libplexus-component-annotations-java libplexus-interpolation-java
  libplexus-sec-dispatcher-java libplexus-utils2-java libpthread-stubs0-dev
  libsisu-inject-java libsisu-plexus-java libslf4j-java libsm-dev
  libwagon-file-java libwagon-http-shaded-java libwagon-provider-api-java
```

Fig. 3: Installation of openjdk-17-jdk, maven and git

The installations are verified with version commands. Java and javac both report version 17.0.20.1, and Maven reports that it is using Java 17, so there is no Java-version mismatch.

```
java -version
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ java -version
openjdk version "17.0.20.1" 2026-08-18
OpenJDK Runtime Environment (build 17.0.20.1+1-1-24.04-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.20.1+1-1-24.04-Ubuntu, mixed mode, sharing)
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig. 4: Java runtime version - OpenJDK 17.0.20.1

```
javac -version
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ javac -version
javac 17.0.20.1
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig. 5: Java compiler version - javac 17.0.20.1

```
mvn -version
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$ mvn -version
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 17.0.20.1, vendor: Ubuntu, runtime: /usr/lib/jvm/java-17-openjdk-a
md64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "7.0.0-42-generic", arch: "amd64", family: "unix"
sk-naser-hussain@sk-naser-hussain-VirtualBox:~$
```

Fig. 6: Apache Maven 3.8.7 running on Java 17.0.20.1

```
git --version
```

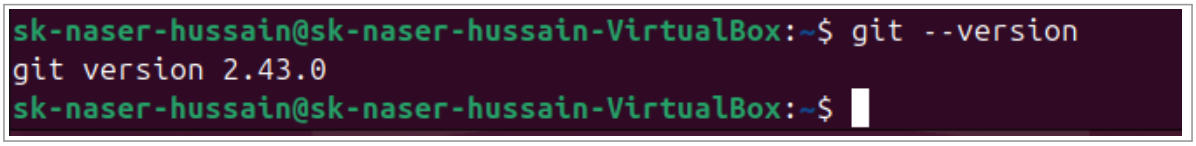


Fig. 7: Git version 2.43.0

Step 3: Create the CloudSim Plus Maven Project

The project directory and the standard Maven source folder are created (*-p* creates missing parent directories), and the terminal is moved into the project root.

```
mkdir -p ~/CloudSchedulingLab/src/main/java/lab
```

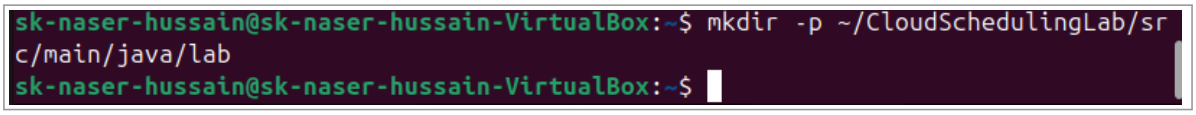


Fig. 8: Creating the project folder structure

```
cd ~/CloudSchedulingLab
```

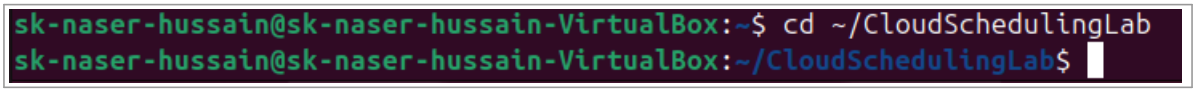


Fig. 9: Changing into the project root directory

The **pom.xml** file defines the Maven project, sets Java 17 as the compiler target and declares CloudSim Plus 8.5.5 as a dependency, which Maven downloads automatically.

```
nano pom.xml
```



Fig. 10: pom.xml created in the nano editor

Step 4: Resolve Dependencies

Maven downloads CloudSim Plus and its supporting libraries into the local repository, and *mvn clean* removes any old build output.

```
mvn dependency:resolve
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~/CloudSchedulingLab$ mvn dependency:resolve
[INFO] Scanning for projects...
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.pom (8.1 kB at 3.5 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-resources-plugin/2.6/maven-resources-plugin-2.6.jar (30 kB at 83 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/2.4/maven-jar-plugin-2.4.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/2.4/maven-jar-plugin-2.4.pom (5.8 kB at 23 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/2.4/maven-jar-plugin-2.4.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/2.4/maven-jar-plugin-2.4.jar (34 kB at 98 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.pom (10 kB at 24 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.1/maven-compiler-plugin-3.1.jar
```

Fig. 11: Maven downloading plugins and the CloudSim Plus dependencies

```
mvn clean
```

```
sk-naser-hussain@sk-naser-hussain-VirtualBox:~/CloudSchedulingLab$ mvn clean
[INFO] Scanning for projects...
[INFO]
[INFO] ----------------------< lab:cloud-scheduling-lab >----------------------
[INFO] Building cloud-scheduling-lab 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ cloud-scheduling-lab ---
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0/plexus-utils-3.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0/plexus-utils-3.0.pom (4.1 kB at 2.2 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-api/2.0.6/maven-plugin-api-2.0.6.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0/plexus-utils-3.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-plugin-api/2.0.6/maven-plugin-api-2.0.6.jar (13 kB at 21 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0/plexus-utils-3.0.jar (226 kB at 161 kB/s)
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  5.252 s
[INFO] Finished at: 2026-11-08T08:42:27+05:30
[INFO] ------------------------------------------------------------------------
sk-naser-hussain@sk-naser-hussain-VirtualBox:~/CloudSchedulingLab$
```

Fig. 12: mvn clean completed with BUILD SUCCESS

Step 5: Implement the Experiment

The Java source file **SchedulingExperiment.java** is created inside the *lab* package. It builds the datacenter, VMs and cloudlets, and runs the four scheduler combinations one after another. The complete program is given in Section 7.

```
nano src/main/java/lab/SchedulingExperiment.java
```

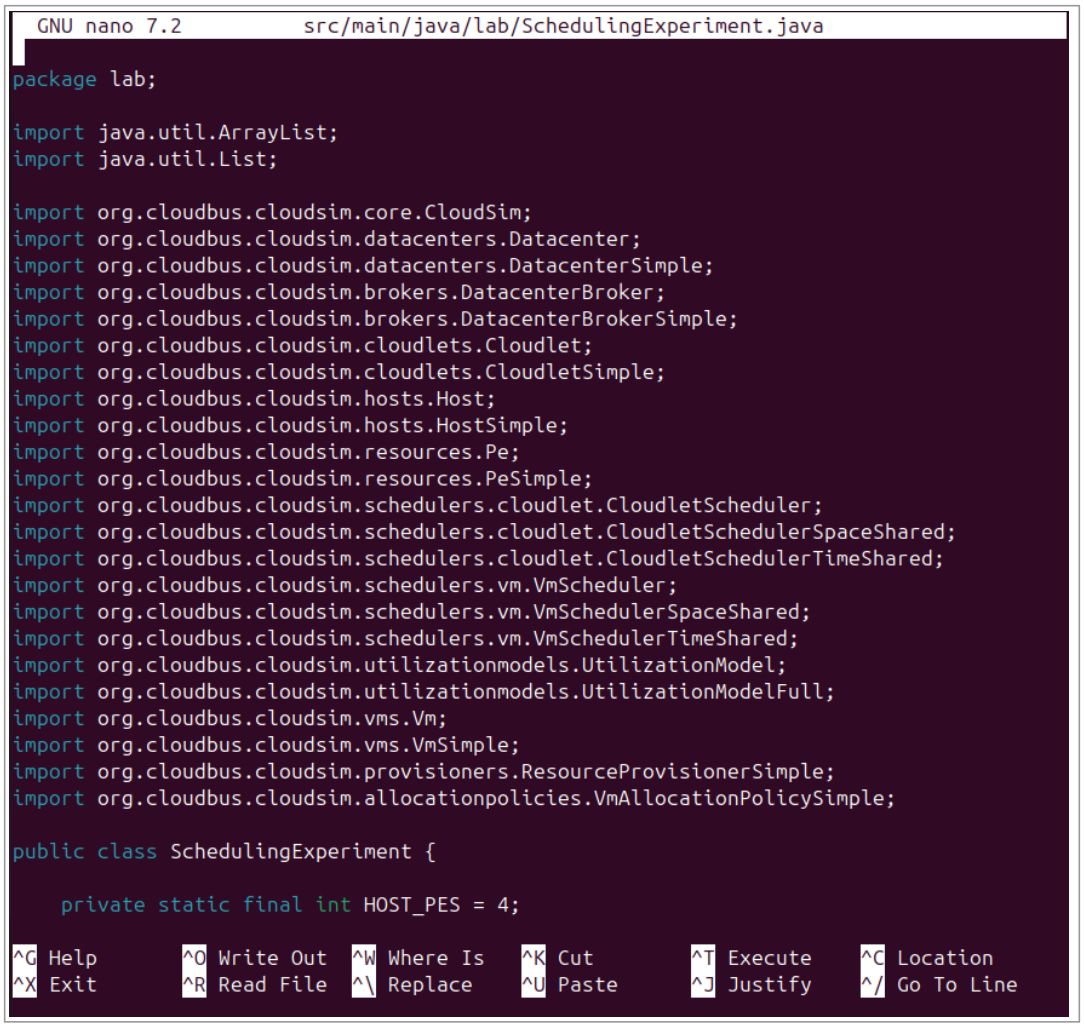


Fig. 13: SchedulingExperiment.java opened in nano (package declaration, imports and constants)

```
ls src/main/java/lab
```

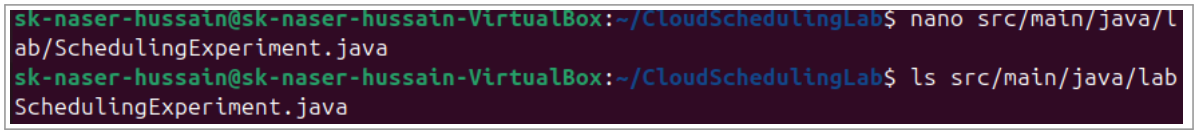


Fig. 14: Confirming that the source file exists in the project

Step 6: Compile and Execute

mvn clean compile compiles the source and reports any syntax or API errors before the simulation is run. The build completed successfully with Java release 17.

```
mvn clean compile
```

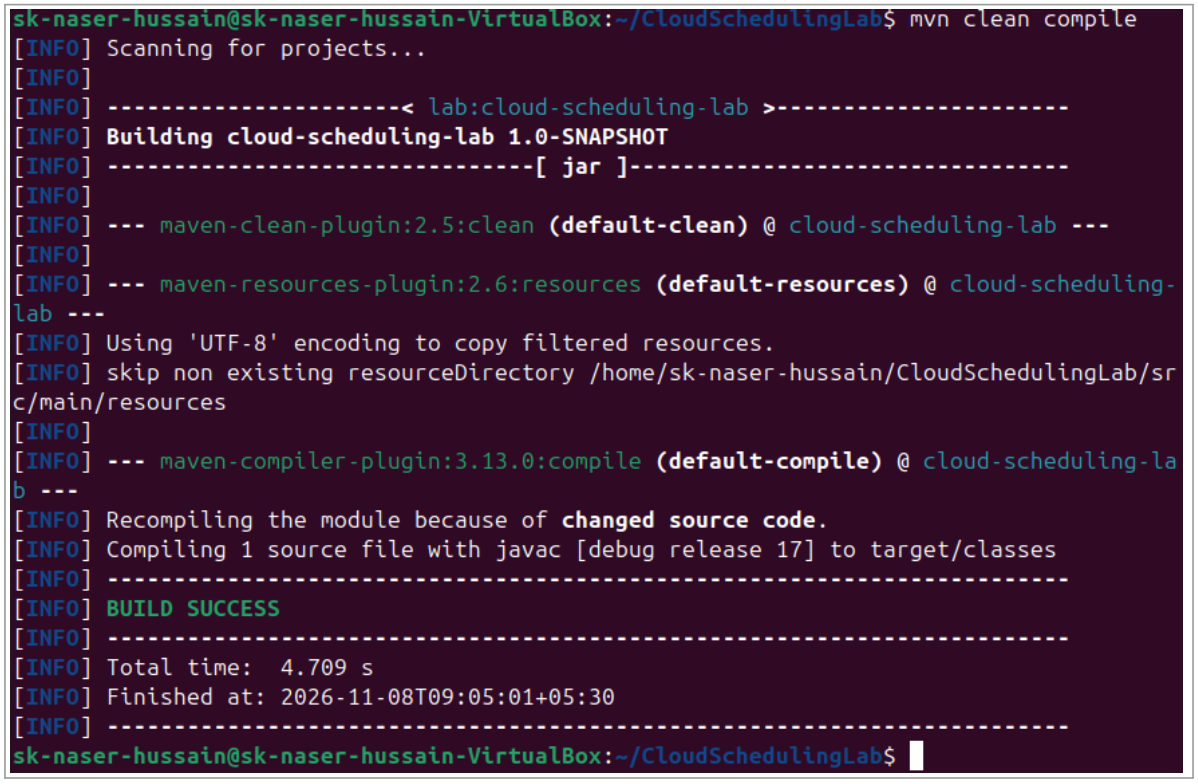


Fig. 15: Successful compilation - 1 source file compiled, BUILD SUCCESS

The simulation is executed with the Maven Exec plugin (version 3.5.0), which runs the *main()* method of the class. The plugin prefix was resolved without needing the fully-qualified fallback command. The run starts with Scenario A (VM SpaceShared + Cloudlet SpaceShared).

```
mvn exec:java
```

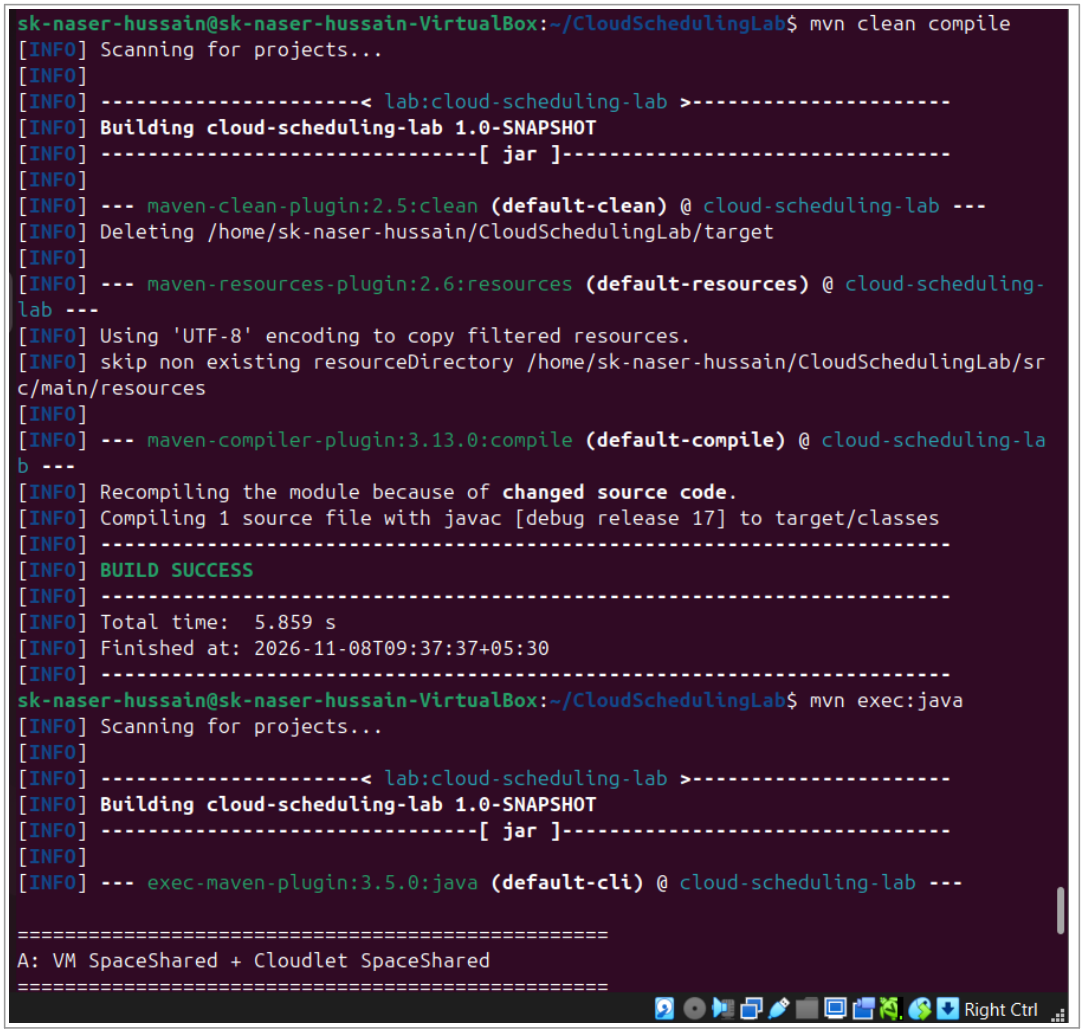


Fig. 16: Compile followed by execution of the simulation - Scenario A begins

6. Result Analysis

In every scenario the same workload is run: 4 cloudlets (20000 MI each, 1 PE each) are submitted to 2 VMs. The broker distributes them round-robin, so each VM receives 2 cloudlets. The key capacities are:

• Host: 4 PEs of 1000 MIPS. The 2 VMs need 2 PEs each, i.e. exactly 4 PEs, so both VMs fit on the host under **both** the space-shared and time-shared VM schedulers.
• Each VM has 2 PEs and each of its 2 cloudlets needs 1 PE, so both cloudlets can execute at the same time under **both** cloudlet schedulers.
• Each cloudlet therefore runs on a full 1000 MIPS core, and its execution time is 20000 MI / 1000 MIPS = **20 seconds**.

Expected behaviour of the four scenarios based on these parameters:

Scenario	VM Sched.	Cloudlet Sched.	Expected behaviour
A	Space	Space	All 4 cloudlets run in parallel, about 20 s each
B	Space	Time	Same as A - no contention, so no slicing effect
C	Time	Space	Same as A - the 4 PEs are fully available to the VMs
D	Time	Time	Same as A - no contention at either level

The scheduling policies differ only when resources are over-subscribed. For example, if more cloudlets than PEs were assigned to a VM, a space-shared cloudlet scheduler would queue the extra cloudlets (they would finish in successive batches), whereas a time-shared scheduler would run them all together at a reduced speed (they would finish at about the same, later time). Likewise, a time-shared VM scheduler would allow more VMs than physical PEs, while a space-shared one would refuse to place them. With the given configuration, the load exactly matches the capacity, which is why the policies do not change the completion pattern.

7. Complete Source Code - SchedulingExperiment.java

```
package lab;

import java.util.ArrayList;
import java.util.List;

import org.cloudbus.cloudsim.core.CloudSim;
import org.cloudbus.cloudsim.datacenters.Datacenter;
import org.cloudbus.cloudsim.datacenters.DatacenterSimple;
import org.cloudbus.cloudsim.brokers.DatacenterBroker;
import org.cloudbus.cloudsim.brokers.DatacenterBrokerSimple;
import org.cloudbus.cloudsim.cloudlets.Cloudlet;
import org.cloudbus.cloudsim.cloudlets.CloudletSimple;
import org.cloudbus.cloudsim.hosts.Host;
import org.cloudbus.cloudsim.hosts.HostSimple;
import org.cloudbus.cloudsim.resources.Pe;
import org.cloudbus.cloudsim.resources.PeSimple;
import org.cloudbus.cloudsim.schedulers.cloudlet.CloudletScheduler;
import org.cloudbus.cloudsim.schedulers.cloudlet.CloudletSchedulerSpaceShared;
import org.cloudbus.cloudsim.schedulers.cloudlet.CloudletSchedulerTimeShared;
import org.cloudbus.cloudsim.schedulers.vm.VmScheduler;
import org.cloudbus.cloudsim.schedulers.vm.VmSchedulerSpaceShared;
import org.cloudbus.cloudsim.schedulers.vm.VmSchedulerTimeShared;
import org.cloudbus.cloudsim.utilizationmodels.UtilizationModel;
import org.cloudbus.cloudsim.utilizationmodels.UtilizationModelFull;
import org.cloudbus.cloudsim.vms.Vm;
import org.cloudbus.cloudsim.vms.VmSimple;
import org.cloudbus.cloudsim.provisioners.ResourceProvisionerSimple;
import org.cloudbus.cloudsim.allocationpolicies.VmAllocationPolicySimple;

public class SchedulingExperiment {

    private static final int HOST_PES = 4;
```

```
private static final int VM_COUNT = 2;
private static final int CLOUDLET_COUNT = 4;
private static final long MIPS = 1000;
private static final long CLOUDLET_LENGTH = 20000;

public static void main(String[] args) {
    runScenario("A: VM SpaceShared + Cloudlet SpaceShared",
            new VmSchedulerSpaceShared(),
            new CloudletSchedulerSpaceShared());

    runScenario("B: VM SpaceShared + Cloudlet TimeShared",
            new VmSchedulerSpaceShared(),
            new CloudletSchedulerTimeShared());

    runScenario("C: VM TimeShared + Cloudlet SpaceShared",
            new VmSchedulerTimeShared(),
            new CloudletSchedulerSpaceShared());

    runScenario("D: VM TimeShared + Cloudlet TimeShared",
            new VmSchedulerTimeShared(),
            new CloudletSchedulerTimeShared());
}

private static void runScenario(String title,
                                VmScheduler vmScheduler,
                                CloudletScheduler cloudletScheduler) {
    System.out.println("\n================================================");
    System.out.println(title);
    System.out.println("================================================");

    CloudSim simulation = new CloudSim();
    createDatacenter(simulation, vmScheduler);

    DatacenterBroker broker = new DatacenterBrokerSimple(simulation);
    List<Vm> vmList = createVms(cloudletScheduler);
    List<Cloudlet> cloudletList = createCloudlets();

    broker.submitVmList(vmList);
    broker.submitCloudletList(cloudletList);

    simulation.start();

    List<Cloudlet> finished = broker.getCloudletFinishedList();

    System.out.printf("%-10s %-10s %-12s %-12s %-12s%n",
            "Cloudlet", "VM", "Status", "Start", "Finish");

    for (Cloudlet cloudlet : finished) {
        System.out.printf("%-10d %-10d %-12s %-12.2f %-12.2f%n",
                cloudlet.getId(), cloudlet.getVm().getId(),
                cloudlet.getStatus(), cloudlet.getExecStartTime(),
                cloudlet.getFinishTime());
    }

    System.out.println("Completed Cloudlets: " + finished.size()
            + " / " + CLOUDLET_COUNT);
}

private static Datacenter createDatacenter(CloudSim simulation,
                                           VmScheduler vmScheduler) {
    List<Pe> peList = new ArrayList<>();
    for (int i = 0; i < HOST_PES; i++) {
        peList.add(new PeSimple(MIPS));
    }

    Host host = new HostSimple(8192, 100000, 1000000, peList);
    host.setRamProvisioner(new ResourceProvisionerSimple());
    host.setBwProvisioner(new ResourceProvisionerSimple());
    host.setVmScheduler(vmScheduler);

    List<Host> hostList = new ArrayList<>();
    hostList.add(host);

    return new DatacenterSimple(simulation, hostList,
            new VmAllocationPolicySimple());
}
```

```
    private static List<Vm> createVms(CloudletScheduler scheduler) {
        List<Vm> vmList = new ArrayList<>();
        for (int i = 0; i < VM_COUNT; i++) {
            Vm vm = new VmSimple(i, MIPS, 2);
            vm.setRam(2048).setBw(10000).setSize(10000);
            vm.setCloudletScheduler(scheduler);
            vmList.add(vm);
        }
        return vmList;
    }

    private static List<Cloudlet> createCloudlets() {
        List<Cloudlet> cloudletList = new ArrayList<>();
        UtilizationModel utilization = new UtilizationModelFull();

        for (int i = 0; i < CLOUDLET_COUNT; i++) {
            Cloudlet cloudlet = new CloudletSimple(
                    CLOUDLET_LENGTH, 1, utilization);
            cloudlet.setSizes(300);
            cloudletList.add(cloudlet);
        }
        return cloudletList;
    }
}
```

8. Conclusion

The Java 17 and Maven environment was set up successfully, the CloudSim Plus 8.5.5 project was created, and the SchedulingExperiment program compiled without errors and started executing the four scheduling scenarios. The experiment demonstrates the two levels of scheduling in a cloud - the VM scheduler at the host level and the cloudlet scheduler at the VM level - and the space-shared and time-shared policies at each level. With 4 PEs on the host, 2 PEs per VM and 1 PE per cloudlet, resources are never over-subscribed, so each cloudlet receives a dedicated core and runs for about 20 seconds regardless of the policy chosen. The policies become visibly different only under resource contention.